

國立臺灣大學電機資訊學院資訊工程學系



碩士論文

Department of Computer Science and Information Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

大語言模型偵測價格操縱攻擊

Detecting Price Manipulation Attacks Using Large  
Language Models

葉富銘

Ye, Fu-Ming

指導教授：蕭旭君 博士

Advisor: Hsu-Chun Hsiao, Ph.D.

中華民國 115 年 7 月

July 2026

國立臺灣大學碩士學位論文

口試委員會審定書



大語言模型偵測價格操縱攻擊

Detecting Price Manipulation Attacks Using Large  
Language Models

本論文係葉富銘君（R13922146）在國立臺灣大學資訊工程學系完成之碩士學位論文，於民國 115 年 7 月 16 日承下列考試委員審查通過及口試及格，特此證明

口試委員：\_\_\_\_\_

（指導教授）

|       |       |
|-------|-------|
| _____ | _____ |
| _____ | _____ |
| _____ | _____ |
| _____ | _____ |

所 長：\_\_\_\_\_





# Acknowledgements

Thanks





## 摘要

隨著去中心化金融快速發展，智慧合約承載的資產規模與金融邏輯日益複雜，其安全性也成為區塊鏈應用能否被信任的關鍵。價格操縱漏洞，尤其是可透過閃電貸放大影響的情境，並不只是單一程式語法錯誤；漏洞是否成立取決於合約如何取得價格、該價格如何影響清算、抵押品估值、鑄幣、會計狀態或其他關鍵金融操作，以及系統是否具備足夠的時間加權平均價格、預言機驗證、滑價限制或其他保護機制。

過往已有許多研究嘗試以交易分析、靜態分析或大型語言模型輔助智慧合約漏洞偵測。然而，既有工作多以攻擊事件、單一合約、協定層級標籤或二元分類結果作為主要評估單位，與真實審計流程仍存在落差。審計人員通常面對的是完整 protocol repository，並需要能指出可疑程式位置、根因、影響、修補方向與支持證據的 finding；若工具產生過多不受支持的警報，即使偵測到正確漏洞，也可能增加人工確認成本。

本研究以協助審計人員快速定位閃電貸價格操縱漏洞為目標，設計並實作一套結合靜態分析與大型語言模型推論的原型架構。系統先從完整 repository 中萃取候選資料流證據與相關程式脈絡，再透過路徑摘要與分組降低重複資訊，最後由分階段大型語言模型流程產生結構化的 audit-style finding。每一項 finding 皆包含可疑程式位置、根因說明、影響描述、缺失的保護機制與支持證據，使審計人員

能回溯工具判斷並進一步驗證。

為了評估該工具是否能服務於審計情境，本研究不僅保留協議層級的精確率、召回率與調和平均分數，也進一步設計報告層級的評估框架。此框架以公開審計報告中的 findings 作為比較目標，從漏洞定位、根因解釋、影響與修補建議一致性、語意相似度，以及過度回報懲罰等面向衡量輸出品質，藉此反映工具對審計人員的實際幫助與可能造成的額外負擔。

初步結果與原型觀察顯示，此工作流程能將大量原始資料流證據收斂為較少且可追溯的 audit-style findings，並揭露單純二元分類指標難以呈現的失敗模式，例如錯誤的程式定位、過多不受支持的報告，以及根因解釋偏移。本研究的主要貢獻在於將大型語言模型輔助智慧合約漏洞偵測的評估重心，從是否偵測到漏洞推進到是否能幫助審計人員在完整 repository 中快速定位並理解閃電貸價格操縱漏洞。

**關鍵字：**去中心化金融、智慧合約安全、價格操縱、閃電貸、大型語言模型、靜態污點分析、審計輔助

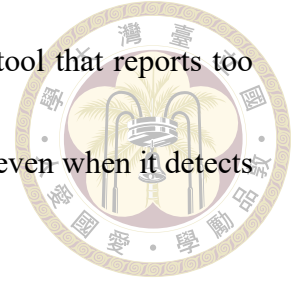


# Abstract

Decentralized finance (DeFi) has made smart contracts responsible for increasingly complex financial logic and high-value assets. As a result, smart contract security has become central to the reliability of blockchain applications. Flash-loan price manipulation is a particularly challenging vulnerability class because the risk cannot be determined from syntax alone. Whether a vulnerability exists depends on how a contract obtains price information, how that value affects liquidation, collateral valuation, minting, accounting, or other financial operations, and whether sufficient protection such as time-weighted pricing, oracle validation, slippage checks, or protocol-specific safeguards is applied.

Prior research has explored transaction analysis, static analysis, and large language models (LLMs) for smart contract vulnerability detection. However, many evaluations still focus on attack events, individual contracts, protocol-level labels, or binary classification results. This does not fully match real audit workflows, where auditors inspect an entire protocol repository and expect findings that identify suspicious code locations, root

causes, impacts, mitigation strategies, and supporting evidence. A tool that reports too many unsupported issues may still increase manual verification cost even when it detects a real vulnerability.



This thesis therefore studies flash-loan price manipulation detection as an audit-assistance problem. It designs and implements a prototype workflow that extracts candidate data-flow evidence and relevant code context from a repository, reduces redundant paths into representative evidence, and uses a staged LLM workflow to generate structured audit-style findings. Each finding is designed to report the suspicious code location, root cause, impact, missing protection, and supporting evidence so that auditors can trace and verify the tool’s judgment.

The evaluation design combines protocol-level detection metrics with report-level measurements. In addition to precision, recall, and F1-score, the framework compares generated reports against public audit findings using localization agreement, root-cause explanation quality, impact and mitigation alignment, semantic similarity, and an over-reporting penalty that reflects manual review cost.

Preliminary prototype observations suggest that the workflow can reduce large sets of raw data-flow evidence into fewer traceable audit-style findings while exposing failure modes that binary metrics tend to hide, including incorrect code localization, unsupported

extra reports, and root-cause drift. The main contribution of this thesis is to shift the evaluation of LLM-assisted smart contract security tools toward whether they help auditors quickly localize and understand flash-loan price manipulation vulnerabilities in complete repositories.

**Keywords:** Decentralized Finance, Smart Contract Security, Price Manipulation, Flash Loans, Large Language Models, Static Taint Analysis, Audit Assistance





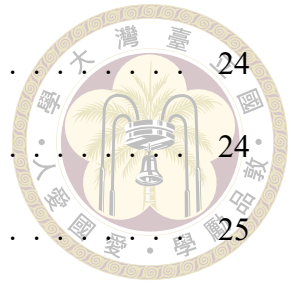
# Contents

|                                                         | Page |
|---------------------------------------------------------|------|
| Verification Letter from the Oral Examination Committee | i    |
| Acknowledgements                                        | iii  |
| 摘要                                                      | v    |
| Abstract                                                | vii  |
| Contents                                                | xi   |
| List of Figures                                         | xv   |
| List of Tables                                          | xvii |
| Denotation                                              | xix  |
| Chapter 1 Introduction                                  | 1    |
| Chapter 2 Background and Related Work                   | 5    |
| 2.1 Price Manipulation in DeFi . . . . .                | 5    |
| 2.2 Smart Contract Auditing Practice . . . . .          | 6    |
| 2.3 Transaction-Based and Post-Hoc Detection . . . . .  | 7    |
| 2.4 Code-Based and Pre-Mortem Detection . . . . .       | 7    |
| 2.5 LLM-Assisted Smart Contract Analysis . . . . .      | 8    |
| 2.6 Audit Reports as Evaluation Material . . . . .      | 9    |
| 2.7 Limitations of Binary Metrics . . . . .             | 10   |

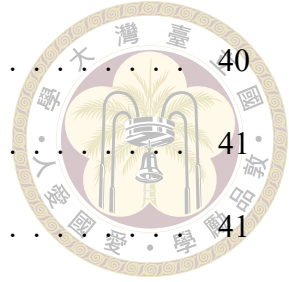


|                  |                                               |           |
|------------------|-----------------------------------------------|-----------|
| <b>Chapter 3</b> | <b>Design</b>                                 | <b>13</b> |
| 3.1              | Design Goals . . . . .                        | 13        |
| 3.2              | Workflow Overview . . . . .                   | 14        |
| 3.3              | Static Analysis Design . . . . .              | 15        |
| 3.3.1            | Source Model . . . . .                        | 15        |
| 3.3.2            | Sink Model . . . . .                          | 15        |
| 3.3.3            | Propagation and Path Reconstruction . . . . . | 16        |
| 3.4              | Representative Path Design . . . . .          | 16        |
| 3.5              | LLM Analysis Design . . . . .                 | 17        |
| 3.5.1            | Stage 0: Source Classification . . . . .      | 17        |
| 3.5.2            | Stage 1: Sink Group Triage . . . . .          | 17        |
| 3.5.3            | Stage 2: Root-Cause Judgment . . . . .        | 18        |
| 3.6              | Evaluation Design . . . . .                   | 18        |
| 3.6.1            | Ground Truth . . . . .                        | 18        |
| 3.6.2            | Report Matching . . . . .                     | 18        |
| 3.6.3            | Over-Reporting Penalty . . . . .              | 19        |
| 3.7              | Scope . . . . .                               | 19        |
| <b>Chapter 4</b> | <b>Implementation</b>                         | <b>21</b> |
| 4.1              | Prototype Overview . . . . .                  | 21        |
| 4.2              | Analyzer Orchestration . . . . .              | 21        |
| 4.3              | Source and Sink Configuration . . . . .       | 22        |
| 4.4              | Taint Engine . . . . .                        | 23        |
| 4.5              | Path Tracking and Grouping . . . . .          | 23        |

|                  |                                              |           |
|------------------|----------------------------------------------|-----------|
| 4.6              | Prompt Generation . . . . .                  | 24        |
| 4.7              | LLM Engine . . . . .                         | 24        |
| 4.8              | Dataset Utilities . . . . .                  | 25        |
| 4.9              | Execution Modes . . . . .                    | 26        |
| 4.10             | Current Implementation Limitations . . . . . | 26        |
| <b>Chapter 5</b> | <b>Evaluation</b>                            | <b>29</b> |
| 5.1              | Evaluation Goals . . . . .                   | 29        |
| 5.2              | Dataset Plan . . . . .                       | 29        |
| 5.3              | Metrics . . . . .                            | 30        |
| 5.3.1            | Binary Metrics . . . . .                     | 30        |
| 5.3.2            | Localization Metrics . . . . .               | 31        |
| 5.3.3            | Semantic Explanation Score . . . . .         | 32        |
| 5.3.4            | Over-Reporting Penalty . . . . .             | 32        |
| 5.4              | Matching Procedure . . . . .                 | 33        |
| 5.5              | Baselines . . . . .                          | 33        |
| 5.6              | Preliminary Observations . . . . .           | 34        |
| 5.6.1            | Spartan . . . . .                            | 34        |
| 5.6.2            | Predy . . . . .                              | 35        |
| 5.6.3            | Abracadabra Money . . . . .                  | 35        |
| 5.7              | Threats to Validity . . . . .                | 36        |
| 5.8              | Expected Outcome . . . . .                   | 36        |
| <b>Chapter 6</b> | <b>Conclusion and Future Work</b>            | <b>39</b> |
| 6.1              | Conclusion . . . . .                         | 39        |



|                                  |                                           |           |
|----------------------------------|-------------------------------------------|-----------|
| 6.2                              | Current Status . . . . .                  | 40        |
| 6.3                              | Future Work . . . . .                     | 41        |
| 6.3.1                            | Complete Sanity Verification . . . . .    | 41        |
| 6.3.2                            | Expand Sink Coverage . . . . .            | 41        |
| 6.3.3                            | Freeze a Build-Verified Dataset . . . . . | 41        |
| 6.3.4                            | Calibrate Semantic Matching . . . . .     | 42        |
| 6.3.5                            | Implement Stronger Baselines . . . . .    | 42        |
| 6.3.6                            | Human Auditor Study . . . . .             | 42        |
| 6.4                              | Final Remarks . . . . .                   | 43        |
| <b>References</b>                |                                           | <b>45</b> |
| <b>Appendix A — Introduction</b> |                                           | <b>47</b> |
| A.1                              | Introduction . . . . .                    | 47        |
| A.2                              | Further Introduction . . . . .            | 47        |
| <b>Appendix B — Introduction</b> |                                           | <b>49</b> |
| B.1                              | Introduction . . . . .                    | 49        |
| B.2                              | Further Introduction . . . . .            | 49        |





## List of Figures





## List of Tables





# Denotation

|            |                                                                                              |
|------------|----------------------------------------------------------------------------------------------|
| AMM        | Automated Market Maker.                                                                      |
| API        | Application Programming Interface.                                                           |
| CFG        | Control-Flow Graph.                                                                          |
| CSV        | Comma-Separated Values.                                                                      |
| DeFi       | Decentralized Finance.                                                                       |
| EVM        | Ethereum Virtual Machine.                                                                    |
| F1-score   | The harmonic mean of precision and recall.                                                   |
| IR         | Intermediate Representation.                                                                 |
| JSON       | JavaScript Object Notation.                                                                  |
| LLM        | Large Language Model.                                                                        |
| PMDetector | A prior hybrid framework for detecting price manipulation vulnerabilities in DeFi protocols. |

|                                 |                                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| RQ                              | Research Question.                                                                                                           |
| SSA                             | Static Single Assignment.                                                                                                    |
| Slither                         | A static analysis framework for Solidity smart contracts.                                                                    |
| SlithIR                         | The intermediate representation exposed by Slither.                                                                          |
| TWAP                            | Time-Weighted Average Price.                                                                                                 |
| $p$                             | A project in the over-reporting penalty, or a predicted explanation in the semantic explanation score, depending on context. |
| $g$                             | A ground-truth audit-finding explanation in the semantic explanation score.                                                  |
| $E(\cdot)$                      | The embedding function used to map report text into a vector representation.                                                 |
| $\tau$                          | The semantic similarity threshold used when computing the description score.                                                 |
| $\cos(E(p), E(g))$              | Cosine similarity between the predicted explanation embedding and the ground-truth explanation embedding.                    |
| $\text{DescriptionScore}(p, g)$ | The semantic explanation score between a predicted report and a ground-truth finding.                                        |
| $\text{Penalty}(p)$             | The over-reporting penalty for project $p$ .                                                                                 |
| $\#\text{Predictions}(p)$       | The number of generated findings for project $p$ .                                                                           |



#GroundTruth( $p$ )

The number of relevant ground-truth audit findings for project  $p$ .







# Chapter 1 Introduction

Decentralized finance (DeFi) protocols encode financial logic directly in smart contracts. A lending protocol determines whether a position is liquidatable from collateral prices, an automated market maker updates liquidity and swap states from pool values, and a vault accounts for deposits, debt, and rewards from asset balances and reserve data. When these values are read from manipulable spot prices, stale or misconfigured oracles, or insufficiently protected price helper functions, attackers can temporarily distort the protocol's view of value and induce incorrect economic behavior. As a result, price manipulation has become an important and difficult vulnerability class in smart contract security.

Flash-loan price manipulation is especially difficult to reason about because the vulnerability is rarely a simple syntactic bug. The same price read may be safe or unsafe depending on how it is computed, how fresh it is, what protection surrounds it, and which financial decision later depends on it. For example, using a current pool price directly in collateral valuation or liquidation logic can be dangerous, while a properly configured time-weighted average price (TWAP), independent oracle check, deviation bound, or slippage control may make a similar-looking code pattern acceptable. Likewise, an accounting update, token transfer, or reward calculation is only security-critical when the manipulated value can influence an irreversible economic state transition. This depen-

dence on economic context makes price manipulation hard for detectors that only match fixed code patterns.



Prior work has made substantial progress in detecting DeFi attacks and smart contract vulnerabilities. Some systems analyze historical transactions to find attacks that already occurred. Others inspect smart contract code or bytecode to detect vulnerable patterns before exploitation. Recent approaches also combine static analysis with large language models (LLMs) to reason about code context and economic intent. These directions are useful, but their problem setting and evaluation often remain centered on attack detection, individual contracts, protocol-level vulnerability labels, or binary classification metrics.

Real audits have a different shape. Auditors typically inspect an entire protocol repository rather than a single function or isolated transaction. They must understand how contracts, helper libraries, price adapters, accounting logic, and external integrations interact. The final audit report is also not a binary label. A useful finding identifies the affected code location or snippet, explains the root cause, describes the impact, recommends mitigation, and provides enough evidence for the auditor and developer to verify the issue. A tool that only says that a protocol is vulnerable may still leave most of the audit work unresolved. A tool that reports many plausible but unsupported issues may increase manual verification cost even if one of its reports is correct.

This thesis therefore studies flash-loan price manipulation detection as an audit-assistance problem. The goal is not to replace human auditors, but to help them quickly locate and understand suspicious price-dependent logic in a complete repository. A useful assistant should gather concrete code evidence, reduce redundant paths, explain why a particular price dependency is risky, and produce findings whose structure resembles the

information auditors already use: code location, root cause, impact, missing protection, and supporting evidence.



To explore this goal, this thesis designs and implements a prototype workflow for Solidity projects. The workflow uses Slither-based static analysis [6] to collect candidate data-flow evidence related to price dependencies and financial operations. It then groups redundant evidence into representative entries and passes the reduced evidence to a staged LLM workflow. The LLM stages are constrained to reason over the extracted evidence and to emit structured findings rather than unconstrained prose. Each final finding is designed to be traceable back to static-analysis artifacts, so that auditors can inspect the evidence rather than trust a free-form model judgment.

This thesis also proposes an evaluation perspective aligned with audit workflows. Protocol-level precision, recall, and F1-score are still useful as baseline measurements, but they do not show whether a generated report points to the right code, explains the right root cause, recommends the right mitigation, or avoids excessive false positives. To evaluate audit usefulness, this thesis uses public audit findings from sources such as Solodit, Code4rena, and Sherlock as report-level comparison targets [1, 2, 5]. The evaluation considers localization, semantic explanation similarity, impact and mitigation agreement, and over-reporting penalties that approximate manual review burden.

In particular, this thesis is organized around three research questions. First, can a static-analysis-plus-LLM workflow generate audit-style findings that help auditors locate flash-loan price manipulation vulnerabilities in a repository? Second, what failure modes are hidden when evaluation stops at binary vulnerability classification? Third, can report-level evaluation based on audit findings better reflect audit usefulness than protocol-level

precision and recall alone?

The contributions of this thesis are as follows:

1. It frames flash-loan price manipulation detection as an audit-assistance problem, emphasizing repository-level localization, explanation quality, and false-positive burden rather than only vulnerable or benign classification.
2. It presents a prototype workflow that combines static evidence extraction, path reduction, staged LLM reasoning, and structured audit-style finding generation for Solidity repositories.
3. It proposes an audit-finding-based evaluation methodology that compares generated reports with public audit findings using localization, root-cause explanation, impact and mitigation alignment, semantic similarity, and over-reporting metrics.

The remainder of this thesis is organized as follows. Chapter 2 reviews price manipulation vulnerabilities, smart contract auditing practice, transaction- and code-based detection approaches, LLM-assisted smart contract analysis, and audit-report-based evaluation. Chapter 3 describes the design of the proposed workflow and evaluation framework. Chapter 4 explains the implementation of the prototype. Chapter 5 presents the planned evaluation methodology and preliminary observations from existing artifacts. Chapter 6 concludes the thesis and discusses future work.



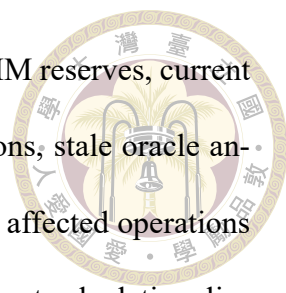


# Chapter 2 Background and Related Work

## 2.1 Price Manipulation in DeFi

Price manipulation vulnerabilities occur when a protocol uses a price-related value that can be influenced by an attacker. In DeFi systems, price is not only used for swaps. It can affect collateral valuation, liquidation eligibility, minting and burning, reward accounting, vault share calculation, margin requirements, protocol-owned liquidity management, and other economic state transitions. A short-lived price deviation may be enough to extract value if the protocol reads the manipulated value and immediately commits an irreversible state change.

Many price manipulation attacks rely on flash loans or low-cost temporary liquidity movement. The attacker changes the apparent price in a pool, triggers a price-dependent operation, and then restores the market state before the end of the transaction. Because the manipulated state can be transient, detecting the vulnerability requires understanding how a price value is obtained, how it is consumed by financial logic, and whether the surrounding implementation contains adequate protection. Merely checking whether a contract imports an oracle interface is not enough.



Common risky price dependencies include direct reads from AMM reserves, current pool ticks, current square-root prices, direct spot price helper functions, stale oracle answers, and protocol-specific wrappers around these values. Common affected operations include collateral checks, liquidation paths, margin updates, swap output calculation, liquidity minting, reward release, and accounting state writes. A safe implementation usually applies mitigation such as TWAP, independent oracle cross-checks, staleness checks, deviation bounds, slippage checks, or delayed settlement. Therefore, the same syntactic pattern can be safe in one context and unsafe in another.

## 2.2 Smart Contract Auditing Practice

Smart contract auditing is usually performed at the level of a full protocol repository. Auditors inspect multiple contracts, helper libraries, external interfaces, deployment assumptions, tests, and business logic. For DeFi protocols, this means that an auditor often needs to trace how an asset price is computed, how that value crosses contract boundaries, and where it influences economically sensitive state transitions.

The output of an audit is also richer than a vulnerability label. A typical finding includes a title, severity, affected code location or snippet, root cause, impact, proof-of-concept context or supporting evidence, and mitigation recommendation. These elements matter because developers need to reproduce and fix the issue, and auditors need to justify why the reported behavior is risky. An automated tool that aims to assist audits should therefore be evaluated by whether it can produce findings with similar information structure, not only by whether it labels a protocol as vulnerable.

## 2.3 Transaction-Based and Post-Hoc Detection



Transaction-based systems analyze on-chain behavior to detect attacks that have already occurred or suspicious transactions that resemble known attack patterns. DeFiRanger recovers high-level DeFi semantics from raw Ethereum transactions by constructing cash-flow trees and lifting low-level transfers into actions such as trades and liquidity operations [8]. It then detects direct and indirect price manipulation attacks using patterns expressed over the recovered semantics. This line of work shows that DeFi price manipulation requires reasoning about financial behavior rather than only code syntax.

DeFiScope extends this post-hoc direction by using LLM reasoning to abstract price calculations from smart contract source code and combine them with high-level DeFi operations recovered from transaction data [9]. It targets both standard and custom price models and reports strong attack-detection results on real-world transactions. However, the primary unit remains attack or transaction detection. Such systems are valuable for monitoring and incident discovery, but they do not directly answer whether an auditor reviewing a repository before deployment can obtain a precise audit-style finding with code location, root cause, impact, and mitigation.

## 2.4 Code-Based and Pre-Mortem Detection

Code-based approaches inspect source code or bytecode before exploitation. This direction is closer to audit assistance because it can reason about a protocol without waiting for an attack transaction. Static analysis is attractive in this setting because deployed smart contract code is transparent and many vulnerability patterns leave structural traces in the

program. Slither is a widely used static analysis framework for Solidity. It parses contracts, constructs control-flow and call information, and exposes SlithIR, an intermediate representation suitable for custom analyses [6].



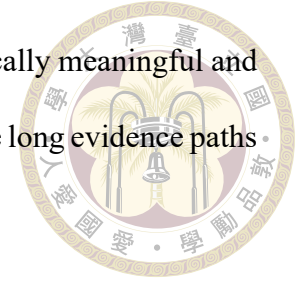
Taint analysis is a common static-analysis technique. In this thesis, source and sink are analysis abstractions: a source is a program value chosen as a starting point for propagation, and a sink is a program location where reaching that value may be relevant to a security question. For price manipulation, this abstraction can model whether a price-like value reaches a financial operation. However, the abstraction is only a tool model. The vulnerability itself still depends on economic meaning, missing protection, and exploitability.

Recent pre-mortem systems apply program analysis and LLMs to price manipulation. PriceSleuth identifies core price-calculation logic, uses an LLM to locate price calculation statements, and applies dependency and propagation analysis to assist price-manipulation detection [7]. PMDetector combines formal attack modeling, static taint analysis, LLM-based filtering and attack simulation, and static validation for DeFi price manipulation vulnerabilities [3]. These works move closer to proactive detection, but their reported evaluation is still mainly organized around detection performance over vulnerable and benign contracts or protocols. This thesis instead asks whether the generated output is useful as an audit finding when an auditor reviews a complete repository.

## 2.5 LLM-Assisted Smart Contract Analysis

Large language models can inspect code snippets and produce security reasoning in natural language. For smart contract analysis, this ability is useful because many findings

depend on explaining why a particular price dependency is economically meaningful and why a mitigation is missing or insufficient. LLMs can also summarize long evidence paths and generate audit-style descriptions.

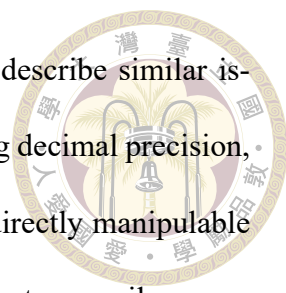


At the same time, LLMs are not reliable as unconstrained detectors. They can hallucinate vulnerabilities, overlook subtle mitigations, or produce plausible but unsupported explanations. This motivates hybrid designs where static analysis supplies concrete evidence and the LLM is constrained to reason over that evidence. Structured output further reduces ambiguity by forcing the model to return machine-checkable fields. The remaining challenge is not only whether the LLM-assisted system detects a vulnerability, but whether its report is localized, evidence-backed, and low-noise enough to help auditors.

## 2.6 Audit Reports as Evaluation Material

Many smart contract vulnerabilities are documented in public audit reports and contest findings. Platforms such as Solodit aggregate findings from audit firms and contests, while Code4rena and Sherlock publish contest reports and issue discussions [1, 2, 5]. These findings often include a title, severity, affected protocol, vulnerability description, impact, proof-of-concept context, and mitigation recommendation.

Audit findings are valuable for this thesis because they resemble the target output of an audit assistant. If a tool claims that it helps auditors, then its output should be compared with what auditors actually write: affected functions, root causes, impacts, and mitigations. Historical exploit labels are useful, but they are often too coarse. They identify that an attack happened; they may not provide enough structure to evaluate whether a generated report localizes the right code path.



Using audit findings also introduces noise. Different auditors describe similar issues using different language. Some reports classify stale price, wrong decimal precision, or oracle misconfiguration as oracle issues even when they are not directly manipulable in a flash-loan setting. Some findings have available source code but no easily reproducible build. A robust evaluation must therefore separate collected findings, source-code-available findings, and build-verified findings.

## 2.7 Limitations of Binary Metrics

Precision, recall, and F1-score are standard metrics for vulnerability detection. They are easy to compute and useful for comparing classifiers. For audit assistance, however, they are incomplete. A binary true positive can hide several report-level errors.

First, a detector may classify a protocol as vulnerable but identify the wrong code region. This still helps less than a report that points to the actual price-dependent logic involved in the finding. Second, a detector may produce too many vulnerable groups. If only one group is correct, binary scoring may still reward the tool while ignoring the extra manual review cost. Third, a detector may identify the correct code area but explain the wrong root cause or recommend the wrong mitigation. An auditor would then need to reinterpret the report before it becomes useful.

These problems motivate report-level evaluation. A generated report should be judged by localization accuracy, root-cause explanation, impact and mitigation agreement, semantic similarity to the ground-truth explanation, and the number of unsupported extra reports. Audit report language is diverse, so exact text matching is too brittle. Two reports may describe the same missing TWAP mitigation using different words. A semantic

similarity score can capture meaning overlap better than lexical overlap, but it must be used together with localization and mitigation checks. This thesis therefore treats binary metrics as a baseline rather than the final measure of success.







## Chapter 3 Design

### 3.1 Design Goals

The goal of this thesis is to design a price manipulation analysis workflow whose output can be evaluated as an audit report, not only as a binary vulnerability label. This leads to four design goals.

**Evidence first.** The system should not ask an LLM to find vulnerabilities from an entire repository without structure. Static analysis must first provide concrete source, sink, and path evidence.

**Semantic grouping.** Raw taint paths are often redundant. A single root cause may produce many syntactic paths because of SSA variables, helper functions, tuple returns, or multiple sink operands. The system should group paths into representative semantic units before LLM analysis.

**Structured reasoning.** LLM output should be constrained to explicit fields such as source classification, sink impact decision, root-cause location, missing mitigation, and supporting evidence IDs. Free-form prose alone is difficult to validate and score.

**Audit-oriented evaluation.** The final output should be comparable to audit findings.

The evaluation should measure localization, explanation match, and over-reporting, not only protocol-level detection.



## 3.2 Workflow Overview

The proposed workflow contains four major phases:

1. Static taint analysis identifies candidate flows from price-related sources to economically meaningful sinks.
2. Path abstraction canonicalizes sources, records provenance, groups sink operands, and emits representative paths.
3. LLM-based analysis filters retained evidence and generates structured audit-style findings.
4. Report-level evaluation matches predictions against audit findings and computes binary, localization, semantic, and over-reporting metrics.

This design follows the high-level direction of PMDetector, but changes the evaluation target. PMDetector-style taint analysis is used as the front end, and LLM reasoning is used as the semantic filter. The difference is that this thesis treats the generated report as the main artifact.



### 3.3 Static Analysis Design

#### 3.3.1 Source Model

The source model intentionally over-approximates possible price-related values. Candidate sources include oracle-like functions, AMM reserve and price reads, external view or pure calls with price-like usage, flash-loan-related calls, and storage reads from price-like state variables. This broad definition is necessary because DeFi protocols often wrap price reads behind protocol-specific helper functions.

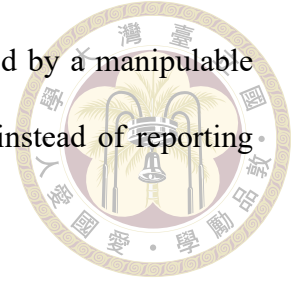
Each source is represented not only as a raw variable but also as a canonical source record. A canonical source groups multiple exact seed identifiers that refer to the same semantic source. For example, a helper function may return a price that is unpacked into several local variables across different callers. Treating these as unrelated sources would inflate the number of findings. Canonicalization allows the system to report one root cause with multiple supporting paths.

#### 3.3.2 Sink Model

The sink model focuses on economically meaningful operations. Candidate sinks include price calculations, collateral checks, lending and borrowing operations, margin updates, liquidation logic, token minting and burning, transfers, and state writes that affect accounting. The sink model is broader than the final vulnerability definition because static analysis should first collect possible evidence.

For evaluation, it is important to record the exact sink operand. A function may

contain several state writes, but only one operand may be controlled by a manipulable price source. The design therefore tracks semantic sink operands instead of reporting only that a function contains a sink.



### 3.3.3 Propagation and Path Reconstruction

Taint propagation operates over Slither IR. The design propagates taint through assignments, arithmetic operations, references, member accesses, tuple unpacking, return values, internal calls, library calls, and selected inter-contract calls. The analysis also records provenance so that a local wrapper can be connected to the terminal external read.

After propagation, the system reconstructs paths from sink operands back to their sources. The raw path list is expected to contain duplicates and near duplicates. This is acceptable at the static-analysis stage because the goal is to avoid missing potentially relevant flows. Redundancy is reduced in the path abstraction phase.

## 3.4 Representative Path Design

The representative path layer converts raw static-analysis results into compact LLM evidence. It has three responsibilities.

First, it collapses duplicate raw paths that share the same semantic source-sink relationship. This reduces prompt size and prevents the LLM from mistaking path volume for vulnerability severity.

Second, it preserves information needed for audit reporting: source label, source function, source provenance, terminal external reads, sink function, marked sink lines,

state writes, external calls, and path counts. The LLM should see enough context to reason about exploitability without receiving the entire repository.



Third, it assigns stable evidence identifiers. The final LLM report refers to supporting evidence IDs, making it possible to trace each generated finding back to concrete static-analysis artifacts.

## 3.5 LLM Analysis Design

### 3.5.1 Stage 0: Source Classification

The first LLM stage classifies each canonical source. The allowed classifications are `spot_amm_price`, `twap_or_oracle_price`, `protocol_state`, `fee_or_liquidity`, `unrelated`, and `unknown`. The model also outputs a decision: `keep`, `drop`, or `need more context`.

This stage is designed to remove obvious false positives early. For example, pool addresses, DAO addresses, balances, total supply, and fee-growth helpers may be part of a taint path but are not necessarily manipulable price sources.

### 3.5.2 Stage 1: Sink Group Triage

For retained sources, the second LLM stage judges whether each sink group is economically dangerous. A sink is dangerous when the tainted value affects accounting, margin, collateral, liquidation, value transfer, minting, burning, or another price-sensitive operation. It is not dangerous when it is telemetry, metadata, harmless caching, or a value not actually controlled by the retained source.

### 3.5.3 Stage 2: Root-Cause Judgment

The final LLM stage emits at most one finding per canonical source. This design reduces over-reporting by preventing every path from becoming a separate finding. Each output includes a verdict, root-cause location, vulnerable external read, missing mitigation, supporting evidence IDs, duplicate or supporting evidence IDs, and a concise explanation.

The output schema is intentionally audit-like. It should be possible to compare the generated explanation with a ground-truth audit finding and ask whether they describe the same issue.

## 3.6 Evaluation Design

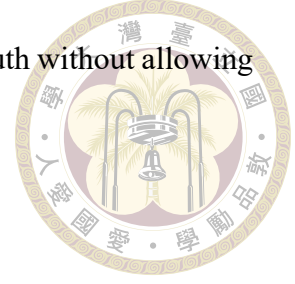
### 3.6.1 Ground Truth

The proposed ground truth is a curated set of audit findings related to price manipulation. Each finding should contain at least the protocol, title, description, affected function or code location when available, impact, and mitigation. The dataset should also record whether source code is available and whether the repository is build-verified.

### 3.6.2 Report Matching

For each project, generated findings are matched against ground-truth audit findings. A pair score combines localization agreement, mitigation agreement, and semantic explanation similarity. Low-scoring pairs are treated as non-matches. Maximum-weight

bipartite matching can then be used to assign predictions to ground truth without allowing one prediction to satisfy multiple findings.



### 3.6.3 Over-Reporting Penalty

The evaluation penalizes unsupported extra findings. For a project  $p$ , the basic penalty is:

$$\text{Penalty}(p) = \max(0, \# \text{Predictions}(p) - \# \text{GroundTruth}(p)).$$

This penalty captures audit workload. A detector that emits many weak reports can increase recall, but it also increases manual review cost.

## 3.7 Scope

The initial scope is price manipulation in Solidity-based DeFi protocols. The dataset may contain related reward or vote manipulation findings, but the core thesis focuses on price-related sources and price-sensitive sinks. Other vulnerability classes such as reentrancy, access control, and bridge-specific logic are outside the main scope unless they directly affect price manipulation mitigation.





## Chapter 4 Implementation

### 4.1 Prototype Overview

The prototype is implemented in Python and built on top of Slither. The main entry point is a price manipulation analyzer that accepts a Solidity file or project directory, initializes Slither, constructs source and sink managers, and runs a project-level taint tracker. The prototype also includes command-line drivers for running taint analysis, generating representative paths, previewing LLM prompts, and executing the staged LLM workflow.

The implementation is organized around three major packages. The `taint_analysis` package contains the static-analysis engine, source and sink definitions, path tracking, path reconstruction, source canonicalization, prompt generation, and Slither integration. The `llm_analysis` package contains the staged LLM engine and prompt templates. The dataset and script directories contain audit-finding collection, repository cloning, build verification, and dataset validation utilities.

### 4.2 Analyzer Orchestration

The main analyzer object is `PriceManipulationAnalyzer`. It receives a contract path and an optional taint-analysis configuration. It then initializes three core components:

- SlitherAnalyzer, which parses the target project and exposes contracts, functions, CFG nodes, and SlithIR operations.
- SourceSinkManager, which manages taint source and sink definitions.
- TaintTracker, which performs source identification, propagation, sink matching, and path reconstruction.



The analyzer returns a dictionary containing the target path, total vulnerable path count, path summaries, serialized paths, number of sources found, and number of sinks found. Each serialized path includes source file and line, source contract and function, source type, canonical source ID, sink file and line, sink kind, sink operand selector, tainted variable, contract, and function.

### 4.3 Source and Sink Configuration

The prototype defines configurable patterns for common price sources and sinks. Oracle patterns include function names such as `latestRoundData`, `getPrice`, `getReserves`, `consult`, and `fetchPrice`. Flash-loan patterns include `flashLoan`, `borrow`, and `executeOperation`. Price calculation sinks include swap and quote helpers, while collateral and lending sinks include liquidation, health factor, borrowing, depositing, withdrawing, minting, burning, and repayment patterns.

The configuration also supports storage-taint sources based on price-like state variable names such as `price`, `rate`, `oracle`, and `feed`. During initialization, the analyzer scans state variables in the target contracts and constructs a storage-read source for matched variables.

## 4.4 Taint Engine



The TaintTracker implements the core static analysis. It maintains a taint map, a path tracker, source records, raw source candidates, sink matches, return summaries, call contexts, and worklist metrics. The engine operates at the project level rather than only within a single contract. This is necessary because DeFi protocols often split price logic, accounting logic, and token operations across multiple contracts and libraries.

The engine identifies raw source candidates and converts them into canonical source records. A source record stores a canonical ID, display label, caller source label, source type, exact seed IDs, provenance chain, terminal external sites, source node, source operation, and related caller metadata. This design allows a source wrapper to remain connected to the underlying price read.

Propagation is delegated to operation-specific handlers. These handlers process assignments, binary operations, calls, type conversions, member accesses, tuple unpacking, and returns. The implementation also maintains return summaries so that a tainted return value can be propagated from a callee back to caller contexts. This is important for helper functions that wrap price reads.

## 4.5 Path Tracking and Grouping

The PathTracker stores vulnerable paths and source aliases. It maps exact variable identifiers to canonical source IDs, tracks reporting seed IDs, and records source realizations created during propagation. This separation between exact IDs and canonical IDs is important because Solidity compilation and Slither SSA can produce many local identi-

fiers for one semantic source.

Raw vulnerable paths are grouped into representative paths before LLM analysis. The grouping layer reduces duplicate paths, merges sink-function records, and preserves marked sink lines. The resulting prompt entries contain source records, sink groups, sink function code, state writes, external calls, representative path counts, raw path counts, and source-sink chains.

This abstraction is a practical requirement. In preliminary runs, one project can produce hundreds of raw paths. Passing all of them directly to an LLM would increase cost and would also make the final report difficult to interpret.

## 4.6 Prompt Generation

The prompt generator renders concise evidence from representative paths. It normalizes whitespace, truncates long statements, formats source provenance, extracts line labels, and renders source-to-sink chains. It also marks sink lines in function bodies so that the LLM can inspect the relevant code without receiving unrelated files.

The generated prompt JSON is saved under a project-specific output directory. This makes each analysis run auditable: the final LLM report can be traced back to the exact prompt entries and static-analysis summary used for that run.

## 4.7 LLM Engine

The LLMEngine orchestrates four named stages: `classify_canonical_sources`, `classify_sources_with_context`, `triage_sink_group_impact`, and `judge_root_cause_findings`.

The engine supports preview mode, which prints prompts without calling an API, and run mode, which calls the configured OpenAI model and writes a structured JSON report.

Every stage validates the model response. Source classification must return one result for every expected source label and must use one of the allowed source classes and decisions. Sink triage must return one result for every expected sink group ID. Root-cause judgment must return one result for every expected canonical source ID and must refer only to known evidence IDs.

This validation does not prove that the model is correct, but it prevents many format and traceability failures. It also makes the report suitable for automatic matching against ground truth.

## 4.8 Dataset Utilities

The prototype includes scripts for collecting and organizing audit findings. The dataset CSV records report date, audit firm, severity, protocol, title, tags, year, compilation status, source-code link, Solodit URL, GitHub link, summary, manipulation label, source-code availability, and remarks. This format supports filtering from a broad collected set to a narrower build-verified evaluation set.

A separate build-manifest verifier checks curated HIGH-severity projects. It validates project IDs, roots, toolchains, build commands, compiler versions, required environment fields, lockfiles, and build output directories. This is important because a finding with a public repository is not automatically usable for static analysis. The project must be available in a reproducible form that Slither can analyze.

## 4.9 Execution Modes



The main taint-analysis test driver can run in three LLM modes:

- `off`: run taint analysis and generate artifacts only.
- `preview`: render the staged LLM prompts without API calls.
- `run`: execute the configured LLM workflow and write a report.

The LLM engine also has a standalone test driver. Given an existing `llm_prompts` JSON file, it can preview prompts, run the workflow, or step through prompts interactively. This separation makes it possible to debug static analysis and LLM reasoning independently.

## 4.10 Current Implementation Limitations

The current implementation is a prototype. It focuses on taint analysis, representative path generation, and staged LLM report generation. Sanity verification rules are not yet fully implemented. In particular, checks for access control, TWAP adequacy, staleness validation, slippage protection, fee-on-transfer behavior, and time-based controls need to be formalized.

The current sink definitions also require expansion. Preliminary artifacts show cases where canonical sources are found but no sink sites are retained. This can happen when the analyzed target does not include the relevant code path, or when the sink model does not cover the protocol's specific economic operation. These limitations are treated as part

of the evaluation rather than hidden.







## Chapter 5 Evaluation

### 5.1 Evaluation Goals

The evaluation is designed to answer whether the prototype produces reports that are useful for auditors. This differs from a pure detector evaluation. A detector evaluation asks whether the system predicts that a project is vulnerable. An audit-usefulness evaluation asks whether the system points to the right code, explains the right root cause, and avoids excessive false reports.

The evaluation therefore has three goals:

1. Measure protocol-level detection performance for compatibility with prior work.
2. Measure report-level correctness, including localization and explanation quality.
3. Measure over-reporting, because unsupported extra findings increase manual audit cost.

### 5.2 Dataset Plan

The planned dataset is built from public audit findings. Each record contains meta-data such as report date, audit firm, severity, protocol, title, tags, year, source-code link,

Solodit URL, GitHub link, summary, whether the issue is classified as price, reward, or vote manipulation, whether source code is available, and remarks.



The dataset should be separated into three subsets.

**Collected findings** include all candidate findings from the collection pipeline. This set is useful for understanding the distribution of protocols, firms, severities, and issue categories, but it may contain entries that are not directly analyzable.

**Source-available findings** include records with public source code. This set is closer to the target evaluation but still may include repositories that are difficult to build or incomplete.

**Build-verified findings** include records whose repositories can be compiled or analyzed reproducibly. This is the primary subset for quantitative evaluation because the static-analysis front end depends on buildable Solidity projects.

This separation avoids overstating the dataset size. A large collected dataset does not automatically imply a large reproducible evaluation set.

## 5.3 Metrics

### 5.3.1 Binary Metrics

The first metric group follows standard vulnerability detection evaluation. A project-level prediction is positive if the system emits at least one price-manipulation finding for the project. Ground truth is positive if the project contains at least one relevant audit finding. From these labels, the evaluation computes true positives, false positives, false

negatives, precision, recall, and F1-score.

These metrics are useful but incomplete. They are reported mainly to make the results comparable with prior work such as PMDetector [3].



### 5.3.2 Localization Metrics

The second metric group evaluates whether a generated report points to the right place. Localization can be measured at several levels:

- Protocol-level localization: the prediction belongs to the same project as the ground-truth finding.
- Contract or function localization: the predicted root-cause location names the same contract, function, wrapper, or semantically equivalent code region.
- Source localization: the predicted vulnerable read corresponds to the same price source or source wrapper.
- Sink localization: the predicted sink corresponds to the same economic operation as the audit finding.
- Mitigation localization: the predicted missing mitigation matches the ground-truth mitigation category.

These levels make partial success visible. For example, a prediction may identify the correct spot price source but miss the exact liquidation sink. Another prediction may identify the affected function but give the wrong mitigation.



### 5.3.3 Semantic Explanation Score

The explanation score compares the generated explanation with the audit-finding summary. The planned implementation embeds both texts using a sentence embedding model and computes cosine similarity:

$$\text{DescriptionScore}(p, g) = \begin{cases} \cos(E(p), E(g)), & \cos(E(p), E(g)) > \tau \\ 0, & \text{otherwise} \end{cases}$$

where  $p$  is the predicted explanation,  $g$  is the ground-truth explanation,  $E(\cdot)$  is the embedding function, and  $\tau$  is the similarity threshold. The current working threshold is 0.7, inspired by semantic evaluation settings such as the Bastet competition [4]. This value must be calibrated on human-labeled prediction-ground-truth pairs before final conclusions are drawn.

### 5.3.4 Over-Reporting Penalty

The over-reporting penalty measures how many extra reports the system emits for a project:

$$\text{Penalty}(p) = \max(0, \# \text{Predictions}(p) - \# \text{GroundTruth}(p)).$$

This penalty is important because the static-analysis phase intentionally over-approximates candidate paths. Without a penalty, the system could improve recall by emitting many

weak findings. In an audit setting, such behavior is costly because each report must be checked manually.



## 5.4 Matching Procedure

For each project, the evaluation builds a pairwise score matrix between predicted reports and ground-truth audit findings. Each pair score combines localization agreement, mitigation agreement, and semantic explanation similarity. Scores below a minimum threshold are set to zero. Maximum-weight bipartite matching is then used to assign predictions to ground-truth findings.

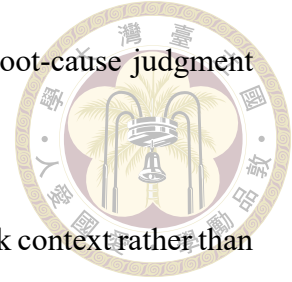
After matching, each prediction is classified as matched or unmatched, and each ground-truth finding is classified as covered or missed. The evaluation reports the total matched score, unmatched predictions, missed findings, binary metrics, and over-reporting penalty.

## 5.5 Baselines

The main baseline is a PMDetector-style binary evaluation of the current workflow. Under this baseline, a project is counted as correctly detected if at least one relevant vulnerability is found. This baseline intentionally ignores whether the report localizes the right path or explains the right root cause.

If time permits, a second baseline should be implemented with a simpler PMDetector-style prompt that emits one report containing `is_high_risk`, vulnerable functions, vulnerable groups, and a reason. Comparing this baseline with the staged workflow would

show whether decomposing source classification, sink triage, and root-cause judgment improves audit-useful output.



The published PMDetector results should be treated as prior-work context rather than a direct baseline unless the dataset and implementation are aligned. The current thesis uses different ground truth and different evaluation metrics.

## 5.6 Preliminary Observations

The current prototype has generated artifacts for several projects. These artifacts are preliminary and should not be interpreted as final evaluation results. They are useful for checking whether the workflow produces meaningful intermediate outputs.

### 5.6.1 Spartan

For the Spartan project, one analysis summary reports 25 analyzed contracts, 67 raw source candidates, 17 canonical sources, 33 sink sites, 217 raw vulnerable paths, and 97 representative paths. The LLM report filters eight source labels down to two retained spot-price sources: `iPOOL.baseAmount` and `iPOOL.tokenAmount`. It then marks 13 sink groups as dangerous and emits two vulnerable root-cause findings.

This result illustrates the intended pipeline behavior. Static analysis generates many raw paths, representative grouping reduces redundancy, and the LLM collapses evidence into a smaller number of root-cause findings. The remaining question for the final evaluation is whether those findings match the corresponding audit or incident ground truth.

### 5.6.2 Predy



For Predy, the LLM report begins from 41 representative paths and six source labels. It keeps `IUniswapV3PoolState.slot0` and `Trade.getSqrtPrice`, drops fee, liquidity, TWAP, and protocol-state sources, and emits two vulnerable root-cause findings. The generated explanations point to spot-price use in `Perp.reallocate` and `Trade.trade`, with missing TWAP or slippage checks.

This result shows the value of source classification. Without the first LLM stage, liquidity and protocol-state values could become false positives. The stage decomposition helps focus root-cause judgment on price-like sources.

### 5.6.3 Abracadabra Money

For one Abracadabra Money run, the summary reports 87 analyzed contracts, 56 raw source candidates, and 23 canonical sources, but zero sink sites and zero representative paths. This is a useful failure case. It may indicate that the current sink definitions do not cover the relevant economic operation, that the selected analysis target does not include the affected code path, or that the issue requires modeling that is not yet implemented.

This case supports the need for transparent reporting. A binary evaluation would count the project as a miss if ground truth contains a relevant finding, but the engineering diagnosis is more specific: source identification may work, while sink coverage fails.

## 5.7 Threats to Validity



**Construct validity.** Semantic similarity may not perfectly represent auditor judgment. The embedding model and threshold must be calibrated against human-labeled pairs.

**Internal validity.** LLM results may depend on model version, prompt wording, temperature, and chunking. Final experiments should fix these parameters and record prompt artifacts.

**External validity.** The thesis focuses on price manipulation in Solidity DeFi protocols. Results may not generalize to other vulnerability classes, non-EVM chains, or protocols with off-chain components.

**Dataset validity.** Audit findings differ in specificity and quality. Some findings are disputed, duplicated, or only partially related to price manipulation. The dataset must document inclusion and exclusion decisions.

**Implementation validity.** The prototype may miss paths because of dynamic dispatch, proxy patterns, incomplete build targets, missing sink definitions, or unsupported Solidity patterns. These failures should be reported rather than hidden.

## 5.8 Expected Outcome

The expected result is not necessarily that the staged workflow achieves the highest binary F1-score. The expected contribution is a clearer measurement of audit usefulness. If a system has good protocol-level detection but poor localization, weak explanation

matching, or high over-reporting, then binary metrics alone are misleading. Conversely, if staged source and sink reasoning improves report-level matching, this would support the thesis that LLM-assisted security tools should be evaluated at the report level.





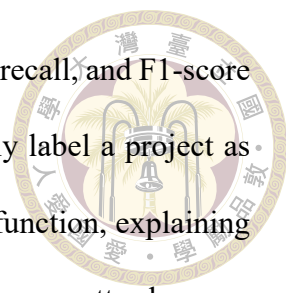


# Chapter 6 Conclusion and Future Work

## 6.1 Conclusion

This thesis studies how PMDetector-style workflows can be evaluated as audit-assistance systems rather than only as binary vulnerability classifiers. Price manipulation vulnerabilities are difficult to detect because the dangerous behavior depends on both data flow and economic context. A price-like source is not always vulnerable, and an economically meaningful sink is not always controlled by a manipulable value. Static analysis can identify possible flows, but it tends to over-approximate. LLMs can reason about code context and missing mitigations, but they require concrete evidence and structured output to be useful.

The prototype developed for this thesis combines these two strengths. It uses Slither-based taint analysis to identify candidate flows, canonical source records to merge equivalent sources, representative path generation to reduce raw path redundancy, and a staged LLM workflow to classify sources, triage sink impact, and emit audit-style root-cause findings. The output is designed to be traceable: each final finding refers back to concrete static-analysis evidence.



The main argument of this thesis is that protocol-level precision, recall, and F1-score are insufficient for evaluating such systems. A detector can correctly label a project as vulnerable while still pointing to the wrong path, naming the wrong function, explaining the wrong mitigation, or producing many extra reports. These failures matter because auditors do not only need a yes-or-no answer. They need precise, actionable, and low-noise reports.

To address this gap, the thesis proposes a report-level evaluation framework based on audit findings. The framework combines binary metrics with localization checks, semantic explanation similarity, mitigation agreement, and an over-reporting penalty. This design is intended to measure whether generated reports are actually useful in an audit workflow.

## 6.2 Current Status

The current implementation should be understood as an initial prototype. It has working components for taint analysis, source and sink tracking, path grouping, LLM prompt generation, staged LLM execution, JSON validation, and preliminary dataset organization. Existing artifacts show that the workflow can produce plausible findings for projects such as Spartan and Predy. They also reveal important limitations, such as missing sink coverage in some projects.

The current dataset work also remains in progress. The collected audit-finding CSV contains useful metadata for filtering and evaluation, and the HIGH dataset slice includes build-readiness checks. However, final quantitative evaluation requires a clearly frozen build-verified subset and manually reviewed ground truth labels.



## 6.3 Future Work

### 6.3.1 Complete Sanity Verification

The most important implementation task is to complete sanity verification. Static reachability and LLM reasoning should be checked against concrete mitigation patterns. Future rules should cover TWAP adequacy, oracle staleness, deviation checks, slippage checks, access control, time-based controls, fee-on-transfer behavior, and protocol-specific guard conditions. These checks would reduce false positives and make LLM findings more defensible.

### 6.3.2 Expand Sink Coverage

Preliminary artifacts show that source identification can succeed while sink matching fails. Future work should expand sink definitions beyond the current patterns. In particular, the analyzer should better capture protocol-specific accounting updates, vault share calculations, liquidity-position updates, reward eligibility checks, and liquidation-adjacent helper functions. Each new sink definition should be tested against real audit findings.

### 6.3.3 Freeze a Build-Verified Dataset

The final evaluation should use a frozen dataset split. Each project should record the repository commit, compiler version, toolchain, build command, required environment variables, and analysis target. Findings should be categorized as price manipulation, related manipulation, or out of scope. Without a frozen dataset, results will be hard to

reproduce and compare.



### **6.3.4 Calibrate Semantic Matching**

The semantic explanation score currently uses a working threshold. Future work should build a small validation set of prediction-ground-truth pairs labeled by human reviewers. The embedding model and threshold should then be calibrated against this validation set. This step is necessary before semantic similarity can be treated as a reliable metric.

### **6.3.5 Implement Stronger Baselines**

A direct PMDetector reproduction is difficult without the original source code, but useful baselines are still possible. One baseline can use the same static analysis front end with a single PMDetector-style prompt that emits `is_high_risk`, vulnerable functions, vulnerable groups, and a reason. Another baseline can evaluate static analysis alone without LLM filtering. Comparing these baselines with the staged workflow would clarify which parts of the design improve report-level usefulness.

### **6.3.6 Human Auditor Study**

The strongest validation would involve human auditors. Given a set of generated reports, auditors could rate whether each report helps them locate the issue, understand the root cause, and decide on mitigation. Such a study would directly evaluate the thesis claim that report-level metrics better capture audit usefulness than binary labels alone.

## 6.4 Final Remarks



LLM-assisted security tools should not be evaluated only by whether they can say that a project is vulnerable. For practical auditing, the harder and more important question is whether they can explain the vulnerability accurately, localize it precisely, and avoid unnecessary noise. This thesis takes a step toward that goal by implementing a PMDetector-inspired prototype and proposing an audit-centered evaluation framework for price manipulation detection.





## References

- [1] Code4rena. Code4rena audits. <https://code4rena.com/audits>, 2026. Accessed 2026-07-09.
- [2] Cyfrin. Solodit. <https://solodit.cyfrin.io/>, 2026. Accessed 2026-07-09.
- [3] L. Liu, W. Zhang, L. Wei, H. Guan, Y. Tian, Y. Liu, and S.-C. Cheung. LLM-powered detection of price manipulation in DeFi. arXiv:2510.21272v2, 2026. Last revised 29 Apr 2026.
- [4] OneSavie. Onesavie bastet competition. <https://www.kaggle.com/competitions/onesavie-bastet/overview>, 2026. Accessed 2026-07-09.
- [5] Sherlock. Sherlock audit contests. <https://audits.sherlock.xyz/>, 2026. Accessed 2026-07-09.
- [6] Trail of Bits. Slither: Static analyzer for Solidity. <https://github.com/crytic/slither>, 2026. Accessed 2026-07-09.
- [7] H. Wu, H. Wang, S. Li, Y. Wu, M. Fan, Y. Zhao, and T. Liu. Detecting state manipulation vulnerabilities in smart contracts using LLM and static analysis. arXiv:2506.08561v2, 2025.

- 
- [8] S. Wu, D. Wang, J. He, Y. Zhou, L. Wu, X. Yuan, Q. He, and K. Ren. DeFiRanger: Detecting price manipulation attacks on DeFi applications. arXiv:2104.15068, 2021.
- [9] J. Zhong, D. Wu, Y. Liu, M. Xie, Y. Liu, Y. Li, and N. Liu. Detecting various DeFi price manipulations with LLM reasoning. arXiv:2502.11521v2, 2025. Accepted by ASE 2025.



# **Appendix A — Introduction**

## **A.1 Introduction**

## **A.2 Further Introduction**





## **Appendix B — Introduction**

### **B.1 Introduction**

### **B.2 Further Introduction**